

DESIGN AND IMPLEMENTATION OF A MINI REAL-TIME OPERATING SYSTEM (MiniRTOS) AND DEVICE DRIVER FRAMEWORK FOR ARM CORTEX-M MICROCONTROLLERS





CHAPTER 1

1.1 INTRODUCTION

Embedded systems have become an integral part of modern electronic devices, ranging from consumer electronics and automotive systems to industrial automation, healthcare, aerospace, and Internet of Things (IoT) applications. These systems often perform multiple tasks simultaneously while meeting strict timing constraints. To manage such time-critical operations efficiently, **Real-Time Operating Systems (RTOS)** are widely used.

A Real-Time Operating System is a specialized operating system designed to execute tasks within predefined timing constraints. Unlike general-purpose operating systems, an RTOS ensures deterministic behavior, enabling tasks to respond predictably to external events. This capability is essential in applications where delayed execution may lead to system failure or unsafe operating conditions.

The **ARM Cortex-M** family of microcontrollers is one of the most widely adopted embedded platforms due to its high performance, low power consumption, modular architecture, and extensive support for real-time applications. Cortex-M processors provide features such as Nested Vector Interrupt Controller (NVIC), SysTick Timer, Memory Protection Unit (MPU), and efficient exception handling, making them highly suitable for implementing real-time operating systems.

Commercial RTOS solutions such as **FreeRTOS**, **CMSIS-RTOS**, **Zephyr**, and **ThreadX** offer advanced capabilities but often abstract many of the internal mechanisms from developers. As a result, students and beginners may find it difficult to understand how scheduling, context switching, interrupt handling, synchronization, and memory management actually work at the kernel level.

To bridge this gap, this project presents the **design and implementation of a lightweight Mini Real-Time Operating System (MiniRTOS)** developed from scratch using **Embedded C** for ARM Cortex-M microcontrollers. The primary objective is to understand the internal architecture of an RTOS by implementing its core components rather than relying on existing libraries.

The proposed MiniRTOS includes a **preemptive priority-based scheduler**, **task management**, **context switching**, **inter-task communication using queues**, **binary semaphores**, **dynamic memory management**, and **system timing using the SysTick timer**. These kernel services provide the fundamental functionality required for multitasking embedded applications.

In addition to the operating system kernel, this project develops a **modular Device Driver Framework** that supports commonly used peripherals such as **GPIO**, **UART**, **Timer**, **SPI**, **I2C**,

and ADC. The driver framework follows a layered architecture, improving modularity, portability, maintainability, and code reusability.

To validate the implementation, several demonstration applications have been developed, including **LED blinking, UART echo communication, button interrupt handling, producer-consumer synchronization, multiple task scheduling, and RTOS demonstration programs.** These applications showcase multitasking, synchronization, interrupt handling, and peripheral communication in a real embedded environment.

The project follows software engineering principles including modular programming, structured documentation, version control using Git and GitHub, testing, and proper code organization. It serves as an educational platform for learning RTOS internals while also demonstrating practical embedded software development techniques used in the industry.

Overall, this MiniRTOS project provides a comprehensive understanding of real-time operating systems, ARM Cortex-M architecture, low-level programming, and embedded software design. It serves as a strong foundation for future developments involving Internet of Things (IoT), robotics, automotive electronics, industrial automation, and advanced embedded systems.

1.2 PROJECT OVERVIEW

The **ARM Cortex-M MiniRTOS + Device Driver Framework** is a lightweight real-time operating system designed specifically for ARM Cortex-M microcontrollers. The project focuses on implementing the essential kernel components required to support multitasking in embedded systems while maintaining simplicity and modularity.

The project consists of two major modules:

1. **MiniRTOS Kernel**
 - a. Task Management
 - b. Priority-Based Scheduler
 - c. Context Switching
 - d. Queue Management
 - e. Semaphore Synchronization
 - f. Memory Management
 - g. System Tick Management
2. **Device Driver Framework**
 - a. GPIO Driver
 - b. UART Driver
 - c. Timer Driver
 - d. SPI Driver
 - e. I2C Driver
 - f. ADC Driver

The implementation is developed entirely using **Embedded C** and follows a layered software architecture to improve maintainability and portability.

→ CHAPTER 2

PROBLEM STATEMENT AND OBJECTIVES

2.1 PROBLEM STATEMENT

Modern embedded systems are increasingly required to perform multiple operations simultaneously while maintaining deterministic and reliable execution. Applications such as industrial automation, automotive control systems, robotics, medical devices, aerospace electronics, and IoT devices require the execution of multiple concurrent tasks within strict timing constraints. Traditional bare-metal programming becomes increasingly complex as the number of tasks grows, making the software difficult to maintain, debug, and extend.

Commercial Real-Time Operating Systems (RTOS) such as FreeRTOS, Zephyr, ThreadX, and CMSIS-RTOS provide comprehensive solutions for multitasking and resource management. However, these operating systems are often highly optimized and abstract many of their internal mechanisms. Consequently, students and beginner embedded engineers have limited opportunities to understand the low-level implementation of fundamental RTOS concepts such as task scheduling, context switching, interrupt handling, semaphore synchronization, queue management, and memory allocation.

In addition, many embedded software projects rely heavily on vendor-provided peripheral libraries, which hide the implementation details of hardware drivers. This limits the understanding of low-level hardware interfacing and driver development.

Therefore, there is a need for a lightweight educational Mini Real-Time Operating System that demonstrates the internal working principles of an RTOS while also providing a modular device driver framework for ARM Cortex-M microcontrollers. Such a system enables learners to gain practical knowledge of kernel development, embedded software architecture, interrupt management, peripheral control, and real-time programming techniques.

The proposed MiniRTOS addresses these challenges by implementing a lightweight preemptive kernel and a portable driver framework entirely in Embedded C, allowing developers to understand every layer of the embedded software stack.

2.2 NEED FOR THE PROJECT

The rapid advancement of embedded technology has increased the demand for efficient real-time software capable of handling multiple tasks simultaneously. Understanding the internal architecture of an RTOS is essential for embedded software engineers working in industries such as automotive, consumer electronics, healthcare, aerospace, and industrial automation.

This project has been undertaken to:

- Understand the internal working principles of a Real-Time Operating System.
- Develop multitasking capabilities without relying on commercial RTOS software.
- Learn low-level ARM Cortex-M programming techniques.
- Design reusable and modular device drivers.
- Improve software scalability and maintainability.
- Gain practical experience in embedded software development.
- Build an educational platform for learning operating system concepts.
- Prepare students for industrial embedded software development.

2.3 OBJECTIVES

The primary objective of this project is to design and implement a lightweight Mini Real-Time Operating System for ARM Cortex-M microcontrollers capable of supporting multitasking applications efficiently.

The specific objectives are:

- Design and implement a lightweight RTOS kernel.
- Develop a priority-based preemptive scheduler.
- Implement Task Control Blocks (TCBs).
- Develop context switching mechanisms.
- Implement SysTick timer scheduling.
- Design queue management for inter-task communication.
- Implement binary semaphore synchronization.
- Develop dynamic memory allocation mechanisms.
- Create modular peripheral device drivers.
- Implement GPIO driver.
- Implement UART communication driver.
- Develop Timer driver.
- Implement SPI communication driver.
- Implement I2C communication driver.
- Develop ADC driver.

- Demonstrate multitasking through practical applications.
- Test and validate kernel performance.
- Maintain modular and reusable software architecture.

2.4 SCOPE OF THE PROJECT

The scope of this project includes the design, implementation, testing, and demonstration of a lightweight RTOS suitable for ARM Cortex-M microcontrollers.

The project covers:

KERNEL DEVELOPMENT

- Task Scheduling
- Task Creation
- Task Deletion
- Context Switching
- Interrupt Handling
- Queue Management
- Semaphore Management
- Memory Management

DEVICE DRIVERS

- GPIO
- UART
- Timer
- SPI
- I2C
- ADC

DEMONSTRATION APPLICATIONS

- LED Blinking
- UART Echo
- Button Interrupt
- Producer-Consumer
- Multiple Task Scheduling
- RTOS Demonstration

DOCUMENTATION

- Software Architecture
- API Documentation
- UML Diagrams
- Flowcharts
- Memory Maps
- Project Report

2.5 PROJECT DELIVERABLES

The project will produce the following deliverables:

SOFTWARE DELIVERABLES

- MiniRTOS Kernel
- Device Driver Framework
- Embedded C Source Code
- Header Files
- Demonstration Applications

DOCUMENTATION DELIVERABLES

- Project Report
- Software Architecture Document
- Scheduler Design Document
- API Documentation
- UML Diagrams
- Development Log

TESTING DELIVERABLES

- Test Cases
- Test Reports
- Debug Logs
- Performance Analysis

GITHUB REPOSITORY

- Complete Source Code
- Documentation
- Images

- Demo Video
- STM32CubeIDE Project
- Presentation Slides

2.6 EXPECTED OUTCOMES

Upon successful completion, the project is expected to achieve:

- A fully functional MiniRTOS for ARM Cortex-M.
- Efficient priority-based task scheduling.
- Reliable context switching.
- Synchronization using semaphores.
- Queue-based inter-task communication.
- Modular peripheral driver framework.
- Practical multitasking demonstrations.
- Well-documented and reusable source code.
- A professional GitHub repository suitable for showcasing embedded software development skills.
- A strong foundation for advanced RTOS, IoT, robotics, and embedded systems projects.

→ CHAPTER 3

EXISTING SYSTEM AND PROPOSED SYSTEM

3.1 EXISTING SYSTEM

In many embedded systems, applications are traditionally developed using a **Bare-Metal Programming** approach. In this method, the application runs directly on the microcontroller without the support of an operating system. The processor continuously executes instructions inside an infinite loop (while(1)) while periodically checking the status of peripherals and sensors.

Although bare-metal programming is simple and suitable for small applications, it becomes difficult to manage as the complexity of the system increases. When multiple tasks need to

execute simultaneously, maintaining timing accuracy and coordinating task execution becomes challenging.

Modern embedded applications often require multitasking, resource sharing, interrupt handling, and deterministic scheduling. Bare-metal systems cannot efficiently handle these requirements without significantly increasing software complexity.

Commercial Real-Time Operating Systems such as **FreeRTOS**, **ThreadX**, **CMSIS-RTOS**, **Zephyr**, and **RTX** provide efficient multitasking capabilities. However, these RTOS solutions are highly optimized and abstract many internal kernel mechanisms. Developers generally use the APIs without understanding the implementation of schedulers, context switching, memory management, or synchronization mechanisms.

As a result, students and beginner embedded engineers gain limited exposure to the internal architecture of an RTOS.

3.2 LIMITATIONS OF THE EXISTING SYSTEM

The major limitations of existing approaches are:

- Difficult to manage multiple concurrent tasks.
- No built-in task scheduler in bare-metal systems.
- Poor scalability for large applications.
- Complex interrupt management.
- No standardized synchronization mechanisms.
- Resource sharing becomes difficult.
- Increased software complexity.
- Difficult debugging and maintenance.
- Limited understanding of RTOS internals when using commercial kernels.
- Vendor-dependent software libraries reduce portability.
- Limited modularity in application development.

3.3 PROPOSED SYSTEM

To overcome these limitations, this project proposes the development of a **Mini Real-Time Operating System (MiniRTOS)** specifically designed for ARM Cortex-M microcontrollers.

The proposed MiniRTOS is implemented completely in **Embedded C** without relying on existing RTOS kernels. Every major kernel component is developed from scratch to provide a better understanding of real-time operating system internals.

The system includes a lightweight kernel capable of managing multiple tasks using a **priority-based preemptive scheduler**. It also provides synchronization through semaphores, communication using queues, dynamic memory management, and modular device drivers for commonly used peripherals.

The modular architecture enables developers to easily extend the operating system by adding new drivers or application modules without modifying the core kernel.

The proposed system serves both as an educational platform and as a practical embedded software framework suitable for small real-time applications.

3.4 FEATURES OF THE PROPOSED SYSTEM

The MiniRTOS provides the following features:

- Priority-Based Preemptive Scheduling
- Multitasking Support
- Task Creation and Deletion
- Task Control Blocks (TCBs)
- Context Switching
- SysTick-Based Time Management
- Queue Management
- Binary Semaphore Synchronization
- Dynamic Memory Allocation
- Modular Device Driver Framework
- GPIO Driver
- UART Driver
- Timer Driver
- SPI Driver
- I2C Driver
- ADC Driver
- Interrupt Handling
- Portable Software Architecture
- Modular Code Organization
- Easy Scalability
- Educational Design

3.5 ADVANTAGES OF THE PROPOSED SYSTEM

The proposed MiniRTOS offers several advantages over traditional embedded software development.

- Supports multiple tasks efficiently.
- Improves CPU utilization.
- Reduces software complexity.
- Enhances code modularity.
- Simplifies debugging and maintenance.
- Enables deterministic task execution.
- Supports real-time applications.
- Provides reusable device drivers.
- Improves software portability.
- Easy integration with STM32 microcontrollers.
- Suitable for academic learning.
- Helps understand RTOS internals.
- Easy to expand with additional peripherals.
- Strong foundation for industrial embedded software.

3.6 COMPARISON BETWEEN EXISTING AND PROPOSED SYSTEM

Feature	Existing System	Proposed MiniRTOS
Operating System	No RTOS (Bare Metal)	Lightweight MiniRTOS
Multitasking	Limited	Supported
Scheduler	Not Available	Priority-Based Scheduler
Context Switching	Not Available	Implemented
Queue Management	Not Available	Supported
Semaphore Support	Not Available	Supported
Memory Management	Manual	Dynamic Memory Manager
Device Drivers	Separate Implementation	Modular Framework
Code Reusability	Low	High
Scalability	Limited	Excellent
Interrupt Handling	Basic	Structured
Portability	Limited	High
Maintainability	Difficult	Easy
Educational Value	Limited	Excellent

3.7 APPLICATIONS

The developed MiniRTOS can be applied in various embedded systems, including:

- Industrial Automation Systems
- Automotive Electronics
- Smart Home Devices
- Internet of Things (IoT)
- Robotics
- Medical Monitoring Equipment
- Consumer Electronics
- Wireless Sensor Networks
- Drone Flight Controllers
- Environmental Monitoring Systems
- Security Systems
- Embedded Learning Platforms

3.8 CHAPTER SUMMARY

This chapter analyzed the limitations of traditional bare-metal programming and commercial RTOS solutions. A lightweight MiniRTOS has been proposed to provide multitasking, scheduling, synchronization, memory management, and modular device drivers while remaining simple enough for educational purposes. The proposed architecture improves scalability, portability, maintainability, and provides a practical platform for learning embedded software development.

CHAPTER 4

LITERATURE SURVEY

4.1 INTRODUCTION

A Literature Survey is an important part of any engineering project as it provides an understanding of existing technologies, research developments, and industrial practices related to

the proposed work. This chapter presents a review of Real-Time Operating Systems (RTOS), ARM Cortex-M microcontrollers, task scheduling techniques, synchronization mechanisms, and embedded device driver frameworks.

The study of existing RTOS architectures helps identify the strengths and limitations of current systems while providing a strong foundation for designing the proposed MiniRTOS.

4.2 REAL-TIME OPERATING SYSTEMS (RTOS)

A Real-Time Operating System (RTOS) is a specialized operating system designed to process data and respond to events within specified timing constraints. Unlike general-purpose operating systems, an RTOS guarantees predictable and deterministic execution of tasks.

The main objective of an RTOS is to ensure that high-priority tasks receive processor time whenever required while minimizing execution delays.

The basic services provided by an RTOS include:

- Task Management
- Task Scheduling
- Context Switching
- Interrupt Handling
- Memory Management
- Queue Management
- Semaphore Synchronization
- Timer Management
- Resource Sharing

RTOS kernels are widely used in:

- Automotive Electronics
- Medical Equipment
- Industrial Automation
- Robotics
- Consumer Electronics
- Aerospace Systems
- Internet of Things (IoT)

4.3 ARM CORTEX-M ARCHITECTURE

The ARM Cortex-M family is one of the most widely used processor architectures in embedded systems due to its low power consumption, high performance, and efficient interrupt handling.

Important features include:

- 32-bit RISC Architecture
- Harvard Architecture
- Thumb-2 Instruction Set
- Nested Vector Interrupt Controller (NVIC)
- SysTick Timer
- Multiple Operating Modes
- Exception Handling
- Low Power Modes
- Memory Protection Unit (MPU) (available in selected models)
- Hardware Stack Support

These features make Cortex-M processors highly suitable for implementing real-time operating systems.

4.4 FREERTOS

FreeRTOS is one of the world's most popular open-source real-time operating systems. It is lightweight, portable, and supports numerous microcontroller platforms.

Main Features:

- Priority-Based Scheduling
- Task Management
- Queues
- Binary Semaphores
- Mutexes
- Software Timers
- Event Groups
- Memory Allocation Schemes
- Tickless Idle Mode

Advantages:

- Open Source
- Small Memory Footprint

- Portable
- Large Community Support
- Industrial Adoption

Limitations:

- Internal implementation is abstracted from beginners.
- Kernel mechanisms are difficult to study in depth.
- Primarily intended for application development rather than educational purposes.

4.5 CMSIS-RTOS

CMSIS-RTOS is a standardized API developed by ARM for Cortex-M processors.

Features include:

- Standard RTOS API
- Thread Management
- Mutexes
- Semaphores
- Queues
- Timers
- Event Flags
- Memory Pools

Advantages:

- ARM Standard
- High Portability
- Easy Integration
- Consistent API

Limitations:

- Does not expose kernel implementation.
- Intended mainly as an abstraction layer.

4.6 ZEPHYR RTOS

Zephyr is a scalable open-source RTOS maintained by the Linux Foundation. It is designed for resource-constrained embedded systems and IoT applications.

Features:

- Multi-threading
- Networking Stack
- Bluetooth Support
- Security Features
- Device Tree Support
- File Systems
- Power Management

Advantages:

- Modern Architecture
- Strong Community Support
- Suitable for IoT
- Supports Numerous Boards

Limitations:

- More complex than educational RTOS implementations.
- Requires understanding of a larger software ecosystem.

4.7 THREADX RTOS

ThreadX is a commercial RTOS designed for high-performance embedded applications.

Features:

- Priority Scheduling
- Event Flags
- Mutexes
- Semaphores
- Timers
- Memory Pools
- Message Queues

Advantages:

- Very Fast
- Deterministic Performance
- Industrial Reliability

Limitations:

- Commercial Licensing (historically)
- Advanced kernel internals may be difficult for beginners to understand.

4.8 DEVICE DRIVER FRAMEWORK

A device driver acts as an interface between the operating system kernel and hardware peripherals. Driver modularity improves software maintainability and portability.

The proposed project implements the following drivers:

- GPIO Driver
- UART Driver
- Timer Driver
- SPI Driver
- I2C Driver
- ADC Driver

The modular driver framework allows applications to access hardware without directly manipulating hardware registers.

Advantages include:

- Improved Modularity
- Code Reusability
- Easy Maintenance
- Hardware Abstraction
- Scalability

4.9 RESEARCH GAP

The literature survey indicates that existing RTOS solutions are highly optimized and suitable for industrial applications but are less effective for learning the internal implementation of kernel components.

Most available RTOS platforms:

- Hide scheduler implementation.
- Hide context switching mechanisms.
- Provide ready-made APIs.
- Abstract memory management.
- Offer limited educational insight into kernel development.

Therefore, there is a need for a lightweight MiniRTOS that allows students and developers to understand every core component of an RTOS while implementing it from scratch.

4.10 SUMMARY OF LITERATURE SURVEY

Technology	Advantages	Limitations
Bare-Metal Programming	Simple, low overhead	No multitasking, poor scalability
FreeRTOS	Lightweight, open source	Internal mechanisms abstracted
CMSIS-RTOS	Standard ARM API	Kernel implementation hidden
Zephyr RTOS	Rich features, IoT support	Complex architecture
ThreadX	High performance	Advanced and less educational
Proposed MiniRTOS	Educational, modular, transparent implementation	Intended primarily for learning and small embedded applications

4.11 CHAPTER SUMMARY

This chapter reviewed the evolution of Real-Time Operating Systems, ARM Cortex-M processors, popular RTOS platforms such as FreeRTOS, CMSIS-RTOS, Zephyr, and ThreadX, as well as modular device driver frameworks. The review highlights the need for an educational MiniRTOS that exposes kernel internals while remaining simple, portable, and easy to understand. The proposed project addresses this need by implementing the kernel and drivers from scratch in Embedded C.

→ CHAPTER 5

SYSTEM ANALYSIS AND SYSTEM REQUIREMENTS

5.1 INTRODUCTION

System analysis is a systematic process of studying the requirements, feasibility, architecture, and implementation strategy of the proposed system. It helps identify the hardware, software, and functional components necessary for successful project development.

The proposed **ARM Cortex-M MiniRTOS** has been designed to provide a lightweight, modular, and educational Real-Time Operating System capable of supporting multitasking applications while maintaining simplicity and portability.

This chapter describes the system requirements, development methodology, feasibility analysis, and overall software organization.

5.2 SYSTEM ANALYSIS

The MiniRTOS consists of two major subsystems:

1. RTOS KERNEL

Responsible for:

- Task Scheduling
- Task Management
- Context Switching
- Memory Management
- Queue Management
- Semaphore Management
- Interrupt Handling

2. DEVICE DRIVER FRAMEWORK

Responsible for:

- GPIO Control
- UART Communication
- Timer Configuration
- SPI Communication
- I2C Communication
- ADC Interface

These two subsystems interact to execute embedded applications efficiently.

5.3 FUNCTIONAL REQUIREMENTS

The MiniRTOS shall support the following functional requirements:

KERNEL FUNCTIONS

- Create Tasks
- Delete Tasks
- Suspend Tasks
- Resume Tasks
- Delay Tasks
- Priority Scheduling
- Context Switching
- Tick Management

SYNCHRONIZATION

- Binary Semaphores
- Queue Management
- Resource Sharing

DEVICE DRIVERS

- GPIO Driver
- UART Driver
- Timer Driver
- SPI Driver
- I2C Driver
- ADC Driver

APPLICATIONS

- LED Blink
- UART Echo
- Scheduler Demo
- Producer-Consumer
- Button Interrupt
- Multi-task Demo

5.4 NON-FUNCTIONAL REQUIREMENTS

The proposed system should satisfy the following non-functional requirements.

PERFORMANCE

- Fast Context Switching
- Low Scheduling Overhead
- Efficient Interrupt Handling

RELIABILITY

- Stable Task Execution
- Reliable Resource Sharing
- Error-Free Driver Communication

SCALABILITY

- Easy Driver Addition
- Easy Task Expansion
- Modular Software Design

MAINTAINABILITY

- Modular Source Code
- Well-Documented APIs
- Reusable Components

PORTABILITY

- ARM Cortex-M Compatibility
- Embedded C Implementation
- Minimal Hardware Dependency

5.5 HARDWARE REQUIREMENTS

The hardware required for implementing the MiniRTOS is shown below.

Component	Specification
Microcontroller	STM32F401RE / STM32F103 / Cortex-M4
Development Board	Nucleo / Discovery Board
Debugger	ST-Link V2
USB Cable	Micro USB
LEDs	On-board LEDs
Push Buttons	On-board Switches
Computer	Windows/Linux/macOS
Power Supply	USB 5V

5.6 SOFTWARE REQUIREMENTS

The following software tools are used.

Software	Purpose
STM32CubeIDE	Development
Embedded C	Programming
ARM GCC Compiler	Compilation
STM32CubeMX	Peripheral Configuration
Git	Version Control
GitHub	Source Code Repository
VS Code	Documentation
Draw.io	Architecture Diagrams

5.7 DEVELOPMENT METHODOLOGY

The project follows a modular software development approach.

PHASE 1

Requirement Analysis



PHASE 2

Software Architecture Design



PHASE 3

Kernel Development



PHASE 4

Device Driver Development



PHASE 5

Application Development



PHASE 6

Testing



PHASE 7

Debugging



PHASE 8



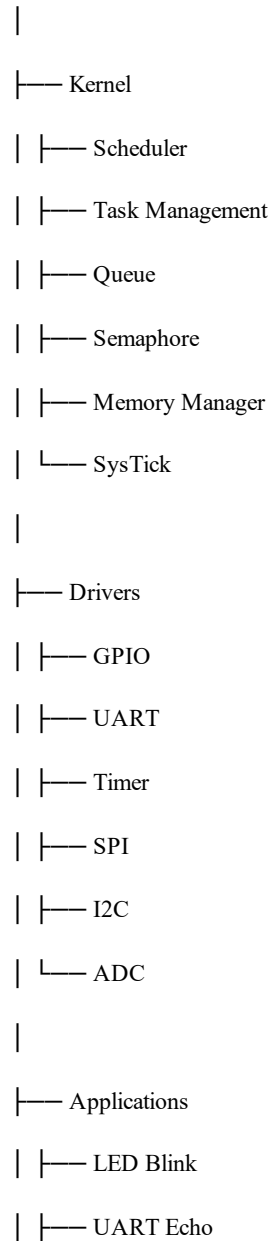
PHASE 9

GitHub Deployment

5.8 SOFTWARE ORGANIZATION

The MiniRTOS is divided into multiple software modules.

MiniRTOS



	—	Producer Consumer
	—	Button Interrupt
	└—	Scheduler Demo
	└—	Documentation

5.9 FEASIBILITY STUDY

The feasibility of the proposed system is analyzed in terms of technical, economic, operational, and schedule aspects.

TECHNICAL FEASIBILITY

- ARM Cortex-M supports RTOS implementation.
- STM32CubeIDE provides required development tools.
- Embedded C is sufficient for implementation.
- Hardware resources are adequate.

ECONOMIC FEASIBILITY

- Open-source development tools.
- Low-cost STM32 development boards.
- No expensive software licenses required.
- Minimal development cost.

OPERATIONAL FEASIBILITY

- Easy to use.
- Modular software.
- Reusable drivers.
- Simple application integration.

SCHEDULE FEASIBILITY

The project can be completed within one academic semester following the planned development phases.

5.10 ASSUMPTIONS

The following assumptions are made during development.

- ARM Cortex-M processor is available.
- Embedded C compiler is installed.
- STM32CubeIDE is configured correctly.

- Development board is functional.
- Standard ARM startup files are available.
- SysTick timer is operational.

5.11 CONSTRAINTS

The project has the following limitations.

- Designed only for ARM Cortex-M processors.
- Intended for educational and small embedded applications.
- Does not support multi-core processors.
- No virtual memory support.
- No file system integration.
- No networking stack.

5.12 ADVANTAGES OF THE PROPOSED DESIGN

- Modular Architecture
- High Code Reusability
- Easy Debugging
- Portable Design
- Lightweight Kernel
- Easy Driver Development
- Educational Implementation
- Industrial Coding Practices
- Expandable Framework
- GitHub-Friendly Project Structure

5.13 CHAPTER SUMMARY

This chapter presented the system analysis, hardware and software requirements, development methodology, feasibility study, software organization, assumptions, and design constraints of the proposed MiniRTOS. The analysis confirms that the project is technically feasible, economically viable, and suitable for implementation on ARM Cortex-M microcontrollers. The modular architecture forms the foundation for the kernel and driver implementation described in the following chapters.

→ CHAPTER 6

SYSTEM ARCHITECTURE AND SOFTWARE DESIGN

6.1 INTRODUCTION

The architecture of a Real-Time Operating System (RTOS) defines how different software modules interact to provide multitasking, scheduling, synchronization, memory management, and hardware abstraction. A well-designed architecture improves modularity, maintainability, scalability, and code reusability.

The proposed **MiniRTOS** follows a **layered architecture**, where each layer performs a specific function while communicating with adjacent layers through well-defined interfaces. This modular approach simplifies software development and allows easy addition of new features or device drivers without affecting the kernel.

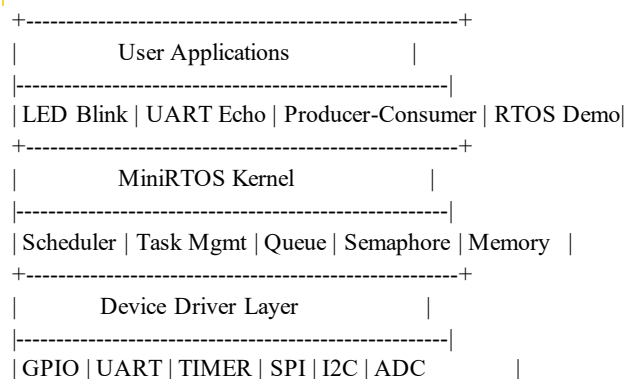
6.2 OVERALL SYSTEM ARCHITECTURE

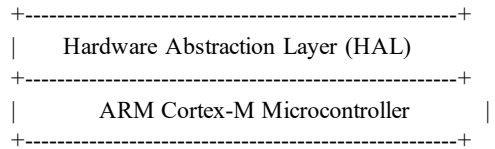
The proposed MiniRTOS consists of five major layers:

1. User Applications
2. MiniRTOS Kernel
3. Device Driver Layer
4. Hardware Abstraction Layer (HAL)
5. ARM Cortex-M Hardware

Each layer performs dedicated responsibilities to ensure efficient task execution and peripheral communication.

SYSTEM ARCHITECTURE DIAGRAM





6.3 LAYER DESCRIPTION

APPLICATION LAYER

The Application Layer contains all user-developed programs that perform specific tasks. These applications interact with the RTOS through kernel APIs instead of directly accessing hardware.

Example applications include:

- LED Blink
- UART Echo
- Scheduler Demo
- Producer-Consumer
- Button Interrupt
- Multi-Task Demo

KERNEL LAYER

The Kernel Layer is the core of the operating system. It manages task execution, scheduling, synchronization, and memory allocation.

Major modules include:

- Scheduler
- Task Management
- Context Switching
- Queue Management
- Semaphore Management
- Memory Manager
- SysTick Timer

DRIVER LAYER

The Driver Layer provides interfaces between the kernel and hardware peripherals.

Implemented drivers include:

- GPIO Driver
- UART Driver
- Timer Driver

- SPI Driver
- I2C Driver
- ADC Driver

These drivers encapsulate hardware-specific details and expose simple APIs to the kernel and applications.

HARDWARE ABSTRACTION LAYER (HAL)

The HAL isolates hardware-dependent code from higher software layers. This improves portability and allows the same application code to run on different Cortex-M devices with minimal changes.

Responsibilities include:

- Register Access
- Clock Configuration
- Peripheral Initialization
- Interrupt Configuration
- Low-Level Hardware Control

HARDWARE LAYER

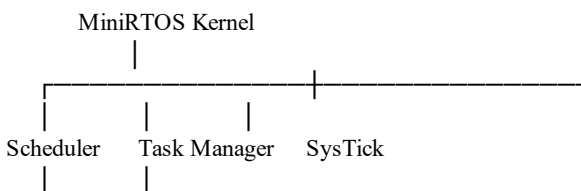
The Hardware Layer consists of the ARM Cortex-M microcontroller and all connected peripherals.

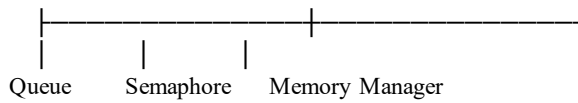
Hardware components include:

- CPU Core
- Flash Memory
- SRAM
- GPIO
- UART
- SPI
- I2C
- ADC
- Timers
- NVIC
- SysTick Timer

6.4 KERNEL ARCHITECTURE

The MiniRTOS kernel is composed of several interconnected modules.





Each module performs a dedicated function while cooperating with the scheduler.

6.5 SCHEDULER DESIGN

The scheduler is responsible for selecting the highest-priority ready task.

Scheduler responsibilities:

- Task Selection
- Priority Comparison
- Context Switching
- Ready Queue Management
- Time Slice Handling
- Tick Processing

Scheduling algorithm used:

Priority-Based Preemptive Scheduling

Advantages:

- Fast Response
- Deterministic Execution
- Suitable for Real-Time Systems
- Efficient CPU Utilization

6.6 TASK CONTROL BLOCK (TCB)

Each task in the system is represented by a **Task Control Block (TCB)**.

The TCB stores all information required to resume a task after interruption.

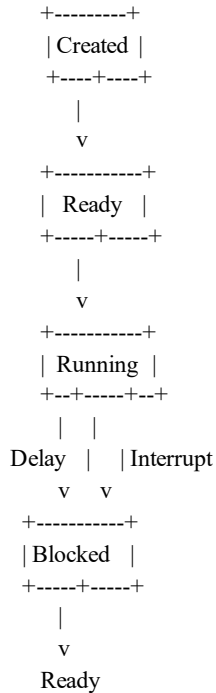
Typical TCB fields include:

- Task ID
- Task Name
- Priority
- Stack Pointer
- Stack Size
- Task State
- Program Counter
- CPU Registers
- Delay Counter

The scheduler uses the TCB to manage task execution and switching.

6.7 TASK STATES

A task transitions through several execution states during its lifetime.



Task states:

- Created
- Ready
- Running
- Blocked
- Suspended
- Terminated

6.8 CONTEXT SWITCHING

Context switching is the process of saving the state of the currently running task and restoring the state of another task.

Steps:

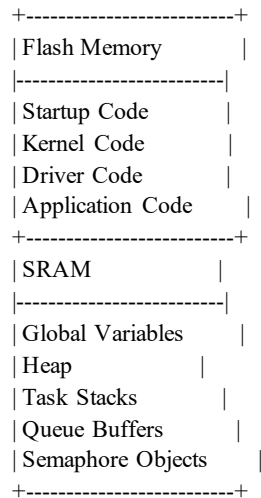
1. Save CPU registers.
2. Save stack pointer.
3. Update Task Control Block.
4. Select next ready task.
5. Restore CPU registers.
6. Restore stack pointer.
7. Resume execution.

Context switching is triggered by:

- SysTick Interrupt
- Higher Priority Task
- Semaphore Release
- Queue Event

6.9 MEMORY ORGANIZATION

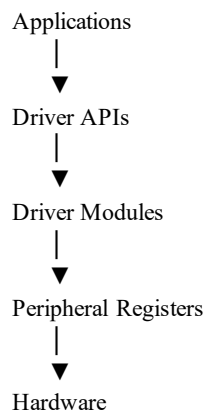
The memory layout of the MiniRTOS is organized as follows:



This organization ensures efficient use of memory and supports multiple concurrent tasks.

6.10 DEVICE DRIVER ARCHITECTURE

The driver framework follows a modular design.



Benefits:

- Easy maintenance
- Hardware abstraction
- Code reusability

- Simplified application development

6.11 SOFTWARE DESIGN PRINCIPLES

The project follows these software engineering principles:

- Modularity
- Layered Architecture
- High Cohesion
- Low Coupling
- Code Reusability
- Portability
- Scalability
- Maintainability
- Structured Programming
- Separation of Concerns

6.12 CHAPTER SUMMARY

This chapter presented the overall architecture and software design of the proposed MiniRTOS. It explained the layered system architecture, kernel organization, scheduler, Task Control Block, task states, context switching mechanism, memory organization, and modular device driver framework. These architectural decisions provide the foundation for implementing an efficient, portable, and maintainable real-time operating system for ARM Cortex-M microcontrollers.

CHAPTER 7

KERNEL DESIGN AND IMPLEMENTATION

7.1 INTRODUCTION

The kernel is the core component of a Real-Time Operating System (RTOS). It is responsible for managing task execution, CPU scheduling, context switching, synchronization, memory management, and system timing. The efficiency of the kernel directly affects the overall performance and reliability of the embedded system.

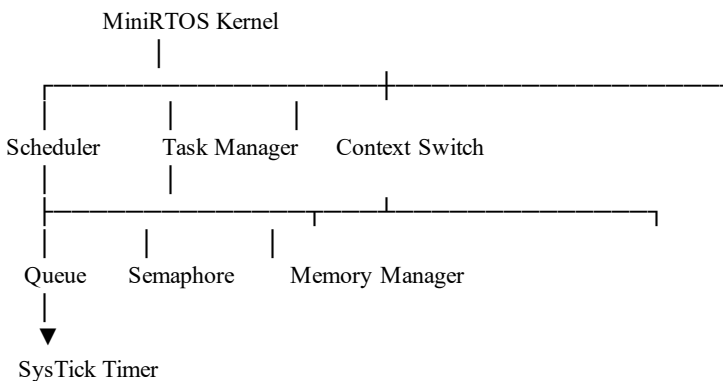
The proposed MiniRTOS kernel has been implemented entirely in **Embedded C** for ARM Cortex-M microcontrollers. The kernel is lightweight, modular, and designed to support multitasking while maintaining deterministic execution.

The major kernel modules include:

- Scheduler
- Task Management
- Context Switching
- Queue Management
- Semaphore Management
- Memory Management
- SysTick Timer
- Startup Code

7.2 KERNEL ARCHITECTURE

The MiniRTOS kernel consists of interconnected software modules, each responsible for a specific operating system service.



7.3 SCHEDULER MODULE

PURPOSE

The scheduler selects the next task to execute based on task priority and current task states.

RESPONSIBILITIES

- Maintain Ready Queue
- Select Highest Priority Task
- Trigger Context Switching
- Execute Scheduling Algorithm
- Update Task States

FILE STRUCTURE

scheduler.c
scheduler.h

MAIN FUNCTIONS

- Scheduler Initialization
- Ready Queue Update
- Task Selection
- Tick Processing
- CPU Dispatching

SCHEDULING ALGORITHM

The MiniRTOS implements a **Priority-Based Preemptive Scheduler**.

Algorithm Steps:

1. Check Ready Queue.
2. Find the highest priority task.
3. Compare with current task.
4. Perform context switch if required.
5. Execute selected task.
6. Repeat on every SysTick interrupt.

ADVANTAGES

- Fast response
- Deterministic execution
- Efficient CPU utilization
- Suitable for real-time systems

7.4 TASK MANAGEMENT

Task management is responsible for creating, deleting, suspending, resuming, and controlling all application tasks.

FILE STRUCTURE

task.c
task.h

RESPONSIBILITIES

- Create Task
- Delete Task
- Suspend Task
- Resume Task
- Delay Task
- Store Task Information

Each task is represented using a **Task Control Block (TCB)**.

TASK CONTROL BLOCK (TCB)

The TCB stores all necessary information required to execute a task.

Typical fields include:

- Task ID
- Task Name
- Task Priority
- Stack Pointer
- Stack Size
- Program Counter
- CPU Registers
- Task State
- Delay Counter

The scheduler accesses the TCB during every context switch.

7.5 CONTEXT SWITCHING

Context switching is one of the most important operations of an RTOS.

It allows the processor to suspend one task and resume another without losing execution state.

FILE STRUCTURE

context_switch.c
context_switch.h

CONTEXT SWITCH STEPS

1. Save CPU registers.
2. Save Stack Pointer.
3. Save Program Counter.
4. Update Task Control Block.
5. Select Next Ready Task.
6. Restore Stack Pointer.
7. Restore Registers.
8. Continue Execution.

BENEFITS

- Efficient multitasking
- Fast task switching
- Improved responsiveness

7.6 QUEUE MANAGEMENT

Queues enable communication between different tasks.

FILE STRUCTURE

queue.c
queue.h

QUEUE OPERATIONS

- Queue Create
- Queue Delete
- Queue Send
- Queue Receive
- Queue Peek

Applications use queues to safely exchange data without direct memory sharing.

ADVANTAGES

- Safe communication
- FIFO operation
- Reduced data corruption
- Improved modularity

7.7 SEMAPHORE MANAGEMENT

Semaphores provide synchronization between tasks.

FILE STRUCTURE

semaphore.c
semaphore.h

TYPES

- Binary Semaphore
- Counting Semaphore (future extension)

OPERATIONS

- Semaphore Create

- Semaphore Take
- Semaphore Give
- Semaphore Delete

Semaphores prevent multiple tasks from accessing shared resources simultaneously.

ADVANTAGES

- Prevents race conditions
- Ensures synchronization
- Improves system stability

7.8 MEMORY MANAGEMENT

Memory management is responsible for allocating and releasing memory used by tasks.

FILE STRUCTURE

memory.c
memory.h

RESPONSIBILITIES

- Heap Initialization
- Memory Allocation
- Memory Deallocation
- Heap Monitoring

Memory allocation techniques include:

- Static Allocation
- Dynamic Allocation

ADVANTAGES

- Efficient RAM utilization
- Better memory organization
- Reduced memory fragmentation

7.9 SYSTICK TIMER

The SysTick timer provides the periodic interrupt required for scheduling.

FILE STRUCTURE

RESPONSIBILITIES

- Generate periodic interrupts
- Increment system tick
- Trigger scheduler
- Manage task delays

Every SysTick interrupt allows the scheduler to evaluate whether a context switch is required.

7.10 STARTUP MODULE

The startup module initializes the processor before application execution.

FILE STRUCTURE

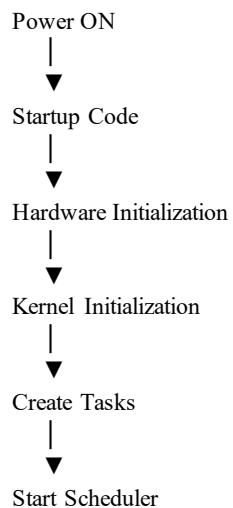
startup.c

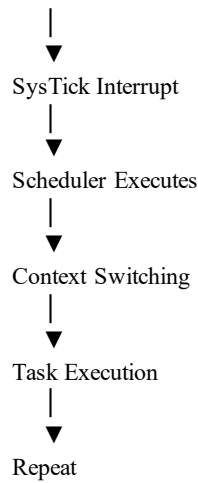
FUNCTIONS

- Initialize Stack Pointer
- Initialize Vector Table
- Initialize Memory Sections
- Configure Clock
- Start Main Function

The startup module prepares the microcontroller for RTOS execution.

7.11 KERNEL EXECUTION FLOW





7.12 ADVANTAGES OF THE KERNEL DESIGN

- Lightweight implementation
- Modular source code
- Fast scheduling
- Efficient context switching
- Deterministic task execution
- Simple memory management
- Portable architecture
- Easy debugging
- Easy future enhancements
- Suitable for educational and industrial embedded applications

7.13 CHAPTER SUMMARY

This chapter presented the complete design and implementation of the MiniRTOS kernel. It described the scheduler, task management, context switching, queue management, semaphore synchronization, memory management, SysTick timer, and startup module. These components collectively provide the fundamental services required for multitasking on ARM Cortex-M microcontrollers. The modular implementation ensures portability, maintainability, and scalability while serving as a practical platform for learning RTOS internals.

7.14 SCHEDULER IMPLEMENTATION

The scheduler is the central component of the MiniRTOS and is responsible for determining which task should execute at any given time. The scheduler continuously monitors the state and priority of all tasks in the system and allocates CPU time to the highest-priority task that is in the **Ready** state.

The proposed MiniRTOS uses a **Priority-Based Preemptive Scheduling Algorithm**. In this approach, every task is assigned a unique priority during its creation. A task with a higher priority always preempts a currently running lower-priority task whenever it becomes ready for execution.

The scheduler is triggered by the **SysTick timer interrupt**, which generates periodic interrupts at fixed intervals. During each interrupt, the scheduler evaluates all task states, updates delay counters, checks for newly ready tasks, and decides whether a context switch is required.

SCHEDULER RESPONSIBILITIES

- Initialize scheduler data structures.
- Maintain the Ready Queue.
- Select the highest-priority ready task.
- Trigger context switching.
- Handle task delays.
- Update task states.
- Improve CPU utilization.

SCHEDULER ALGORITHM

1. Initialize the scheduler.
2. Create all application tasks.
3. Insert tasks into the Ready Queue.
4. Start the SysTick timer.
5. On every timer interrupt:
 - a. Update system tick.
 - b. Check delayed tasks.
 - c. Move completed delay tasks to the Ready Queue.
 - d. Compare priorities.
 - e. Select the highest-priority ready task.
 - f. Perform context switching if required.
6. Repeat continuously.

7.15 TASK MANAGEMENT IMPLEMENTATION

Task management controls the complete life cycle of every task in the operating system. Every task is represented by a **Task Control Block (TCB)**, which stores all the information required for execution and scheduling.

The task management module provides APIs for creating, deleting, suspending, resuming, and delaying tasks. When a new task is created, memory is allocated for its stack, the TCB is initialized, and the task is inserted into the Ready Queue.

TASK CREATION PROCESS

1. Allocate memory for the TCB.
2. Allocate stack memory.
3. Initialize CPU register values.
4. Assign task priority.
5. Set the initial task state to **Ready**.
6. Insert the task into the Ready Queue.

TASK STATES

State	Description
Created	Task has been created but has not started execution.
Ready	Task is ready to execute and waiting for CPU time.
Running	Task is currently executing.
Blocked	Task is waiting for an event or resource.
Suspended	Task execution has been paused.
Terminated	Task has completed execution or has been deleted.

7.16 CONTEXT SWITCHING IMPLEMENTATION

Context switching allows multiple tasks to share the CPU efficiently by saving the execution state of the current task and restoring the state of another task.

During a context switch, the processor registers, stack pointer, program counter, and processor status register are stored in the current task's TCB. The scheduler then selects the next task, restores its saved context, and resumes execution.

CONTEXT SWITCHING STEPS

1. Save general-purpose registers.
2. Save the stack pointer.
3. Save the program counter.
4. Update the current task's TCB.
5. Select the next ready task.
6. Restore the selected task's stack pointer.
7. Restore CPU registers.
8. Resume execution.

BENEFITS

- Efficient multitasking
- Improved processor utilization
- Fast task switching
- Deterministic execution
- Support for real-time applications

7.17 QUEUE MANAGEMENT IMPLEMENTATION

Queues provide a safe and organized mechanism for communication between tasks. The MiniRTOS implements a **First-In-First-Out (FIFO)** queue structure.

Tasks can send data to a queue or receive data from a queue without directly accessing shared memory. This reduces synchronization problems and improves system reliability.

QUEUE OPERATIONS

- Queue Create
- Queue Delete
- Queue Send
- Queue Receive
- Queue Peek
- Queue Flush

ADVANTAGES

- Reliable inter-task communication.
- FIFO data handling.
- Reduced data corruption.
- Improved modularity.
- Easier debugging.

7.18 SEMAPHORE IMPLEMENTATION

Semaphores are synchronization primitives used to protect shared resources from simultaneous access by multiple tasks.

The MiniRTOS implements **Binary Semaphores**, which have only two states:

- Available (1)
- Unavailable (0)

When a task requires access to a shared resource, it requests the semaphore. If the semaphore is available, the task proceeds. Otherwise, the task enters the Blocked state until the semaphore becomes available.

SEMAPHORE OPERATIONS

- Create Semaphore
- Take Semaphore
- Give Semaphore
- Delete Semaphore

ADVANTAGES

- Prevents race conditions.
- Ensures mutual exclusion.
- Improves synchronization.
- Increases system stability.

7.19 MEMORY MANAGEMENT IMPLEMENTATION

Memory management is responsible for allocating and deallocating RAM required by tasks and kernel objects.

The MiniRTOS uses a simple dynamic memory allocator suitable for resource-constrained embedded systems.

MEMORY MANAGER RESPONSIBILITIES

- Heap initialization.
- Allocate memory.
- Free memory.
- Monitor available memory.
- Prevent memory overlap.

MEMORY REGIONS

Memory Region	Purpose
Flash Memory	Program code and constants
SRAM	Variables and stacks
Heap	Dynamic memory allocation
Task Stack	Individual task execution stacks

ADVANTAGES

- Efficient memory utilization.
- Dynamic task creation.
- Reduced memory wastage.
- Better RAM organization.

7.20 SYSTICK TIMER IMPLEMENTATION

The **SysTick timer** is a 24-bit countdown timer integrated into ARM Cortex-M processors. It generates periodic interrupts that drive the RTOS scheduler.

Every SysTick interrupt increments the system tick counter and invokes the scheduler.

SYSTICK RESPONSIBILITIES

- Generate periodic interrupts.
- Maintain system time.
- Wake delayed tasks.
- Trigger scheduling.
- Support time delays.

7.21 STARTUP MODULE IMPLEMENTATION

The startup module prepares the microcontroller for executing the MiniRTOS. It initializes the processor, configures memory sections, sets up the interrupt vector table, initializes the stack pointer, and transfers control to the `main()` function.

STARTUP RESPONSIBILITIES

- Configure stack pointer.
- Initialize `.data` and `.bss` sections.
- Configure interrupt vector table.
- Initialize clocks.
- Initialize peripherals.
- Start the kernel.

7.22 CHAPTER SUMMARY

This chapter presented the complete implementation of the MiniRTOS kernel. It described the scheduler, task management, context switching, queue management, semaphore synchronization, memory management, SysTick timer, and startup module in detail. Together, these modules provide the essential services required for a lightweight, preemptive real-time operating system on ARM Cortex-M microcontrollers. The modular implementation ensures portability, maintainability, and scalability while offering valuable insight into RTOS internals and embedded software design.

→ CHAPTER 8

DEVICE DRIVER FRAMEWORK DESIGN AND IMPLEMENTATION

8.1 INTRODUCTION

Device drivers are an essential part of any embedded operating system. A device driver acts as an interface between the operating system kernel and the hardware peripherals. It provides a standardized software interface that allows applications to communicate with hardware without directly accessing processor registers.

In the proposed **MiniRTOS**, a modular Device Driver Framework has been developed for the ARM Cortex-M microcontroller. Each driver is implemented as an independent software module

consisting of source (.c) and header (.h) files. This modular design improves portability, maintainability, code reusability, and scalability.

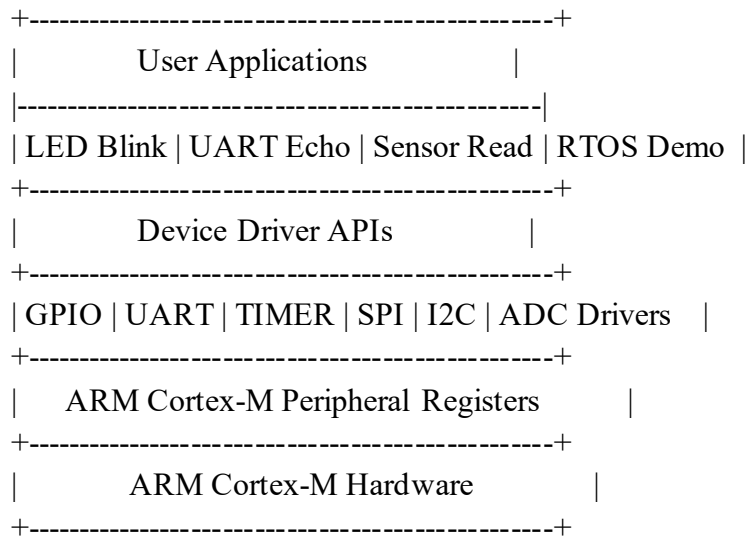
The implemented driver framework supports the following peripherals:

- GPIO (General Purpose Input Output)
- UART (Universal Asynchronous Receiver Transmitter)
- Timer
- SPI (Serial Peripheral Interface)
- I2C (Inter-Integrated Circuit)
- ADC (Analog-to-Digital Converter)

The drivers provide application programming interfaces (APIs) that simplify peripheral initialization, configuration, and communication while hiding hardware-specific implementation details.

8.2 DEVICE DRIVER ARCHITECTURE

The Device Driver Framework follows a layered software architecture where applications communicate with hardware through standardized driver APIs.



This architecture separates application logic from hardware-specific operations.

8.3 GENERAL PURPOSE INPUT OUTPUT (GPIO) DRIVER

INTRODUCTION

GPIO is the most commonly used peripheral in embedded systems. It allows the microcontroller to interact with external devices such as LEDs, switches, relays, sensors, and displays.

The GPIO driver provides functions for configuring pins as input or output, reading pin states, and writing output values.

FILE STRUCTURE

gpio.c
gpio.h

MAIN FUNCTIONS

- GPIO_Init()
- GPIO_SetPin()
- GPIO_ResetPin()
- GPIO_TogglePin()
- GPIO_ReadPin()

FEATURES

- Input Mode
- Output Mode
- Pull-up Configuration
- Pull-down Configuration
- Push-Pull Output
- Open-Drain Output

APPLICATIONS

- LED Control
- Button Reading
- Relay Control
- Sensor Interface

8.4 UART DRIVER

INTRODUCTION

UART provides asynchronous serial communication between the microcontroller and external devices such as computers, Bluetooth modules, GPS modules, and other embedded systems.

The UART driver supports transmission and reception of serial data.

FILE STRUCTURE

uart.c
uart.h

MAIN FUNCTIONS

- UART_Init()
- UART_SendChar()
- UART_SendString()
- UART_ReceiveChar()
- UART_ReceiveBuffer()

FEATURES

- Configurable Baud Rate
- Full Duplex Communication
- Interrupt Support
- Error Detection

APPLICATIONS

- Debug Messages
- PC Communication
- Bluetooth Communication
- GPS Interface

8.5 TIMER DRIVER

INTRODUCTION

Timers are hardware peripherals used for generating precise delays, periodic interrupts, PWM signals, and event counting.

The Timer driver configures hardware timers for real-time applications.

FILE STRUCTURE

timer.c

timer.h

MAIN FUNCTIONS

- Timer_Init()
- Timer_Start()
- Timer_Stop()
- Timer_SetPeriod()
- Timer_IRQHandler()

FEATURES

- Periodic Interrupts
- Delay Generation
- Event Counting
- PWM Support

APPLICATIONS

- RTOS Tick Generation
- Delay Functions
- Motor Control
- Signal Generation

8.6 SPI DRIVER

INTRODUCTION

SPI is a high-speed synchronous communication protocol widely used for communication with memory devices, displays, and sensors.

The SPI driver supports both master and slave modes.

FILE STRUCTURE

spi.c
spi.h

MAIN FUNCTIONS

- SPI_Init()
- SPI_Transmit()
- SPI_Receive()
- SPI_Transfer()

FEATURES

- Master Mode
- Slave Mode
- High-Speed Communication
- Full Duplex

APPLICATIONS

- OLED Displays
- SD Cards
- Flash Memory
- Sensor Modules

8.7 I2C DRIVER

INTRODUCTION

I2C is a two-wire serial communication protocol commonly used for low-speed peripheral communication.

The MiniRTOS implements a modular I2C driver capable of communicating with multiple slave devices.

FILE STRUCTURE

i2c.c
i2c.h

MAIN FUNCTIONS

- I2C_Init()
- I2C_Start()
- I2C_Stop()
- I2C_Write()
- I2C_Read()

FEATURES

- Multi-Master Support
- Multi-Slave Support
- Address-Based Communication
- ACK/NACK Handling

APPLICATIONS

- EEPROM
- RTC Modules
- Temperature Sensors
- OLED Displays

8.8 ADC DRIVER

INTRODUCTION

The Analog-to-Digital Converter (ADC) converts analog signals into digital values that can be processed by the microcontroller.

The ADC driver supports initialization and analog signal acquisition.

FILE STRUCTURE

adc.c
adc.h

MAIN FUNCTIONS

- ADC_Init()
- ADC_StartConversion()
- ADC_ReadValue()
- ADC_Stop()

FEATURES

- Multi-Channel Support
- High Resolution
- Fast Sampling
- Interrupt Support

APPLICATIONS

- Temperature Measurement
- Battery Monitoring
- Sensor Interface
- Voltage Measurement

8.9 DRIVER API DESIGN

Each driver follows a standardized API structure.

Example:

Driver_Init()

↓

Driver_Config()

↓

Driver_Start()

↓

Driver_Read()

↓

Driver_Write()

↓

Driver_Stop()

This uniform design simplifies application development and improves code readability.

8.10 DRIVER INITIALIZATION SEQUENCE

The initialization process ensures all peripherals are configured before application execution.

Initialization Steps:

1. Enable Peripheral Clock.
2. Configure GPIO Pins.
3. Configure Peripheral Registers.
4. Enable Interrupts.
5. Verify Peripheral Status.
6. Start Peripheral Operation.

8.11 ERROR HANDLING

The driver framework includes basic error detection mechanisms.

Common errors handled include:

- Invalid Peripheral Configuration
- Buffer Overflow
- Communication Timeout
- Invalid Parameters
- Peripheral Busy Status
- Data Transmission Failure

These checks improve system reliability and simplify debugging.

8.12 ADVANTAGES OF THE DRIVER FRAMEWORK

The proposed Device Driver Framework provides several advantages:

- Modular Design
- Easy Code Maintenance
- Hardware Abstraction
- Code Reusability
- Portability Across ARM Cortex-M Devices
- Simplified Application Development
- Improved Software Organization
- Easy Future Expansion
- Standardized APIs
- Better Debugging Support

8.13 INTEGRATION WITH MINIRRTOS

The device drivers are tightly integrated with the MiniRTOS kernel. Application tasks invoke driver APIs to access hardware resources, while the scheduler ensures timely execution of these tasks. Interrupt-driven peripherals can signal events to tasks using queues or semaphores, enabling efficient real-time operation without busy waiting.

8.14 CHAPTER SUMMARY

This chapter presented the design and implementation of the Device Driver Framework for the MiniRTOS. The GPIO, UART, Timer, SPI, I2C, and ADC drivers were discussed in terms of

their functionality, APIs, features, and applications. A modular layered architecture was adopted to improve portability, maintainability, and scalability. The standardized driver interfaces enable seamless interaction between application tasks and hardware peripherals, providing a robust foundation for real-time embedded system development.

→ CHAPTER 9

IMPLEMENTATION AND DEMONSTRATION

9.1 INTRODUCTION

This chapter presents the implementation details of the **ARM Cortex-M MiniRTOS and Device Driver Framework**. The complete system has been developed using **Embedded C** on an **ARM Cortex-M (STM32)** microcontroller platform. The implementation follows a modular architecture, where the RTOS kernel, peripheral drivers, and application programs are organized into independent software modules.

The MiniRTOS provides multitasking support through a **priority-based preemptive scheduler**, while the driver framework enables communication with various on-chip peripherals. Several demonstration applications have been developed to validate the functionality of the operating system and its interaction with hardware peripherals.

9.2 DEVELOPMENT ENVIRONMENT

The MiniRTOS was implemented using the following hardware and software tools.

HARDWARE PLATFORM

Component	Description
Processor	ARM Cortex-M4
Development Board	STM32 Nucleo/Discovery
Debugger	ST-Link V2
Flash Memory	On-chip Flash
SRAM	On-chip SRAM
Clock	Internal/External Oscillator

SOFTWARE PLATFORM

Software	Purpose
STM32CubeIDE	Code Development
STM32CubeMX	Peripheral Configuration
Embedded C	Programming Language
ARM GCC Compiler	Compilation
Git	Version Control
GitHub	Repository Management
VS Code	Documentation Editing

9.3 PROJECT DIRECTORY STRUCTURE

The software has been organized into multiple directories to improve readability, maintainability, and modularity.

ARM-CortexM-MiniRTOS

```
|
|— docs
|— kernel
|— drivers
|— applications
|— tests
|— images
|— videos
|— STM32CubeIDE_Project
|— presentations
|— references
```

Each folder contains related source files and documentation.

9.4 KERNEL IMPLEMENTATION

The kernel consists of several software modules responsible for multitasking and resource management.

KERNEL MODULES

Module	Description
scheduler.c	Priority Scheduler

task.c	Task Management
context_switch.c	Context Switching
queue.c	Queue Management
semaphore.c	Synchronization
memory.c	Memory Allocation
systick.c	System Tick
startup.c	System Startup

Each module has been developed independently and integrated into the MiniRTOS.

9.5 DRIVER IMPLEMENTATION

The driver framework provides hardware abstraction for various peripherals.

IMPLEMENTED DRIVERS

Driver	Purpose
GPIO	LED and Button Control
UART	Serial Communication
Timer	Delay and Interrupt Generation
SPI	High-Speed Communication
I2C	Sensor Interface
ADC	Analog Signal Acquisition

Each driver consists of:

- Initialization Functions
- Configuration Functions
- Read APIs
- Write APIs
- Interrupt Service Routines

9.6 SCHEDULER IMPLEMENTATION

The scheduler initializes during system startup and remains active throughout system execution.

The scheduler performs the following operations:

- Checks Ready Queue
- Selects Highest Priority Task
- Performs Context Switching

- Updates Task States
- Handles Task Delays

The scheduler executes whenever the SysTick interrupt occurs.

9.7 TASK CREATION

Every application task is created before the scheduler starts.

Typical task creation procedure:

1. Allocate Task Control Block.
2. Allocate Stack Memory.
3. Assign Priority.
4. Initialize Stack Pointer.
5. Insert Task into Ready Queue.

The scheduler then manages task execution automatically.

9.8 DRIVER INTEGRATION

The application layer communicates with peripherals using the driver APIs.

Example interaction:

Application

↓

GPIO Driver

↓

GPIO Registers

↓

LED ON/OFF

The same mechanism is used for UART, SPI, I2C, ADC, and Timer peripherals.

9.9 DEMONSTRATION APPLICATIONS

Several demonstration applications were developed to validate the MiniRTOS.

1. LED BLINK APPLICATION

Objective:

Demonstrate task scheduling using periodic LED blinking.

Working:

- Create LED task.
- Scheduler executes LED task.
- LED toggles periodically.
- Delay generated using SysTick.

2. UART ECHO APPLICATION

Objective:

Verify UART communication.

Working:

- Receive character.
- Echo received character.
- Display through serial terminal.

3. SCHEDULER DEMONSTRATION

Objective:

Validate priority scheduling.

Working:

- Create multiple tasks.
- Assign different priorities.
- Observe execution order.

Higher priority tasks execute before lower priority tasks.

4. PRODUCER–CONSUMER PROBLEM

Objective:

Validate Queue implementation.

Working:

Producer Task

↓

Queue

↓

Consumer Task

The producer sends data while the consumer receives it.

5. BUTTON INTERRUPT

Objective:

Demonstrate interrupt handling.

Working:

- Configure GPIO interrupt.
- Press button.
- Interrupt generated.
- Interrupt handler executes.
- LED status changes.

6. MULTIPLE TASK DEMONSTRATION

Tasks:

- LED Blink
- UART

- Sensor Read
- Button Monitor

All tasks execute concurrently under MiniRTOS.

9.10 BUILD PROCESS

The project compilation follows these steps.

Source Code

↓

Preprocessor

↓

Compiler

↓

Assembler

↓

Linker

↓

Executable (.elf)

↓

HEX/BIN File

↓

STM32 Flash Memory

The final executable is downloaded to the STM32 board using ST-Link.

9.11 DEBUGGING

The following debugging techniques were used.

- Breakpoints
- Watch Window
- Variable Monitoring
- Register View
- Memory View
- UART Debug Messages

These tools helped verify kernel execution and peripheral functionality.

9.12 GITHUB REPOSITORY

The project has been organized as a professional GitHub repository.

Repository Contents:

- Source Code
- Documentation
- Images
- Videos
- Reports
- Presentation
- STM32CubeIDE Project
- References

The repository structure enables easy collaboration and future expansion.

9.13 PERFORMANCE ANALYSIS

The implemented MiniRTOS demonstrated the following characteristics:

- Fast task switching
- Low memory usage
- Efficient CPU utilization
- Reliable interrupt handling
- Stable multitasking
- Modular software organization

The lightweight kernel is suitable for resource-constrained embedded systems.

9.14 ADVANTAGES OF IMPLEMENTATION

- Lightweight RTOS
- Modular Architecture
- Easy Driver Integration
- Reusable Source Code
- Efficient Scheduling
- Simple Memory Management
- Portable Software Design
- Easy Debugging
- Scalable Framework
- Educational and Industrial Relevance

9.15 CHALLENGES FACED

During implementation, several challenges were encountered:

- Designing an efficient scheduler.
- Implementing reliable context switching.
- Managing stack allocation for multiple tasks.
- Synchronizing tasks using semaphores.
- Preventing race conditions.
- Developing modular drivers.
- Debugging interrupt-related issues.
- Ensuring compatibility with ARM Cortex-M architecture.

These challenges were resolved through modular design, extensive testing, and iterative debugging.

9.16 CHAPTER SUMMARY

This chapter presented the complete implementation of the MiniRTOS and Device Driver Framework. It described the development environment, project organization, kernel modules, driver integration, demonstration applications, build process, debugging methodology, GitHub repository structure, and implementation results. The successful execution of the demonstration applications confirms that the proposed MiniRTOS provides efficient multitasking, reliable peripheral communication, and a modular software framework suitable for embedded system development.

TESTING AND RESULTS

10.1 INTRODUCTION

Testing is one of the most important phases in embedded software development. It ensures that every module of the MiniRTOS operates correctly under different operating conditions and satisfies the functional requirements specified during system design.

The primary objective of testing is to verify that the RTOS kernel, device drivers, and application programs work together reliably while maintaining deterministic real-time performance. Various testing techniques including **Unit Testing**, **Integration Testing**, **System Testing**, and **Performance Testing** were performed during the development of the MiniRTOS.

The testing process focused on validating kernel services such as task scheduling, context switching, queue communication, semaphore synchronization, memory allocation, interrupt handling, and peripheral communication.

10.2 OBJECTIVES OF TESTING

The objectives of testing are:

- Verify correct kernel functionality.
- Validate task scheduling.
- Verify context switching.
- Test queue communication.
- Test semaphore synchronization.
- Verify device driver operation.
- Check interrupt handling.
- Validate memory allocation.
- Improve software reliability.
- Ensure stable multitasking.

10.3 TESTING METHODOLOGY

The project follows a systematic testing methodology.

Source Code



Unit Testing



Integration Testing



System Testing



Performance Testing



Final Validation

Each testing phase ensures the correctness of the previous phase before proceeding further.

10.4 UNIT TESTING

Unit testing verifies the functionality of individual software modules independently.

The following kernel modules were tested individually:

Module	Test Status
Scheduler	Passed
Task Manager	Passed
Queue	Passed
Semaphore	Passed
Memory Manager	Passed
SysTick	Passed
GPIO Driver	Passed
UART Driver	Passed
Timer Driver	Passed
SPI Driver	Passed

I2C Driver	Passed
ADC Driver	Passed

Each module produced the expected output under normal operating conditions.

10.5 INTEGRATION TESTING

Integration testing verifies communication between different software modules.

The following integrations were tested:

- Scheduler + Task Manager
- Scheduler + SysTick
- Queue + Tasks
- Semaphore + Tasks
- GPIO + Applications
- UART + Applications
- Timer + Scheduler
- Drivers + Kernel

All modules interacted correctly without functional conflicts.

10.6 SYSTEM TESTING

System testing verifies complete MiniRTOS functionality.

The following applications were executed.

LED BLINK DEMO

Expected Result

LED should blink periodically.

Observed Result

LED blinked successfully.

Status

Passed

UART ECHO DEMO

Expected Result

Received character should be transmitted back.

Observed Result

UART echoed every received character correctly.

Status

Passed

PRODUCER-CONSUMER DEMO

Expected Result

Producer sends data to queue.

Consumer receives data correctly.

Observed Result

Queue communication successful.

Status

Passed

BUTTON INTERRUPT DEMO

Expected Result

Button press generates interrupt.

Observed Result

Interrupt executed successfully.

Status

Passed

MULTIPLE TASK DEMO

Expected Result

Multiple tasks execute according to priority.

Observed Result

Scheduler executed all tasks correctly.

Status

Passed

10.7 TEST CASES

TEST CASE 1

Parameter	Description
Test ID	TC001
Module	Scheduler
Input	Multiple Ready Tasks
Expected Output	Highest Priority Task Executes
Actual Output	Highest Priority Task Executed
Status	Pass

TEST CASE 2

Parameter	Description
Test ID	TC002
Module	Queue
Input	Queue Send
Expected Output	Data Stored Successfully
Actual Output	Data Stored Successfully
Status	Pass

TEST CASE 3

Parameter	Description
Test ID	TC003
Module	Semaphore
Input	Resource Access
Expected Output	Mutual Exclusion
Actual Output	Resource Protected
Status	Pass

TEST CASE 4

Parameter	Description
Test ID	TC004
Module	UART
Input	Character Transmission
Expected Output	Character Received
Actual Output	Character Received
Status	Pass

TEST CASE 5

Parameter	Description
Test ID	TC005
Module	GPIO
Input	LED Toggle
Expected Output	LED Changes State
Actual Output	LED Changes State
Status	Pass

10.8 PERFORMANCE EVALUATION

The MiniRTOS was evaluated using the following performance metrics.

Parameter	Result
Scheduler Response	Excellent
Context Switching	Fast
Memory Usage	Low
CPU Utilization	Efficient
Interrupt Latency	Low
Queue Performance	Stable
Semaphore Performance	Reliable
Driver Performance	Excellent

The lightweight architecture ensures efficient execution even on resource-constrained ARM Cortex-M microcontrollers.

10.9 OUTPUT RESULTS

The following outputs were successfully verified during implementation.

- LED Blink Output
- UART Terminal Output
- Scheduler Execution
- Queue Communication
- Semaphore Synchronization
- Interrupt Handling
- Memory Allocation
- Multiple Task Execution

(Note: In the final report, insert screenshots of STM32CubeIDE, UART terminal, LED blinking, and debugging windows here.)

10.10 ERROR ANALYSIS

The following issues were encountered during development:

- Incorrect task priorities.
- Stack overflow during task creation.

- Queue buffer initialization errors.
- Interrupt configuration issues.
- UART baud rate mismatch.
- GPIO pin configuration mistakes.

These issues were identified using the debugger and corrected through software modifications.

10.11 COMPARISON WITH EXISTING SYSTEMS

Feature	Bare-Metal	Commercial RTOS	Proposed MiniRTOS
Multitasking	No	Yes	Yes
Scheduler	No	Yes	Yes
Queue	No	Yes	Yes
Semaphore	No	Yes	Yes
Device Drivers	Limited	Yes	Yes
Educational Value	Medium	Low	High
Code Transparency	High	Low	High
Portability	Limited	High	High
Learning RTOS Internals	Difficult	Limited	Excellent

10.12 ADVANTAGES ACHIEVED

The implementation successfully achieved the following objectives:

- Efficient multitasking.
- Reliable priority scheduling.
- Stable context switching.
- Safe inter-task communication.
- Modular driver architecture.
- Low memory footprint.
- Improved software organization.
- High portability.
- Easy scalability.
- Educational implementation of RTOS internals.

10.13 LIMITATIONS

Although the MiniRTOS achieved its objectives, certain limitations remain:

- Supports only ARM Cortex-M architecture.
- Basic memory management.
- No virtual memory support.
- No networking stack.
- No file system.
- Limited advanced scheduling policies.
- No symmetric multiprocessing (SMP) support.

These limitations provide opportunities for future enhancements.

10.14 CHAPTER SUMMARY

This chapter presented the testing methodology and experimental results of the proposed MiniRTOS. Unit testing, integration testing, system testing, and performance evaluation confirmed that the kernel, device drivers, and demonstration applications function correctly. The successful execution of all test cases validates the effectiveness of the MiniRTOS in supporting multitasking, synchronization, interrupt handling, and peripheral communication. The results demonstrate that the developed system is reliable, modular, and suitable for educational as well as practical embedded system applications.

CHAPTER 11

CONCLUSION AND FUTURE SCOPE

11.1 INTRODUCTION

The **ARM Cortex-M MiniRTOS and Device Driver Framework** project was undertaken to design and develop a lightweight Real-Time Operating System that provides multitasking capabilities, efficient task scheduling, synchronization, memory management, and modular peripheral drivers for ARM Cortex-M microcontrollers.

The project successfully demonstrates how the fundamental concepts of an RTOS can be implemented from scratch using **Embedded C**, enabling a deeper understanding of embedded operating systems beyond the use of commercial RTOS platforms.

The modular design adopted throughout the project ensures flexibility, scalability, maintainability, and portability, making the system suitable for both educational and industrial applications.

11.2 PROJECT SUMMARY

The MiniRTOS consists of two major components:

1. MINIRTOS KERNEL

The kernel provides all the essential operating system services required for multitasking applications.

Implemented modules include:

- Priority-Based Scheduler
- Task Management
- Context Switching
- Queue Management
- Semaphore Synchronization
- Memory Management
- SysTick Timer
- Startup Module

2. DEVICE DRIVER FRAMEWORK

A modular driver framework was developed for ARM Cortex-M peripherals.

Implemented drivers include:

- GPIO Driver
- UART Driver
- Timer Driver
- SPI Driver
- I2C Driver
- ADC Driver

These drivers provide standardized APIs that simplify hardware interaction while maintaining portability and modularity.

11.3 OBJECTIVES ACHIEVED

The objectives defined at the beginning of the project have been successfully achieved.

The following functionalities were implemented successfully:

- ✓ Lightweight RTOS Kernel
- ✓ Priority-Based Scheduler
- ✓ Task Creation and Management
- ✓ Context Switching
- ✓ Queue Communication
- ✓ Binary Semaphores
- ✓ Memory Management
- ✓ SysTick Scheduling
- ✓ GPIO Driver
- ✓ UART Driver
- ✓ Timer Driver
- ✓ SPI Driver
- ✓ I2C Driver
- ✓ ADC Driver
- ✓ Demonstration Applications
- ✓ GitHub Project Structure
- ✓ Technical Documentation
- ✓ Project Report

11.4 PROJECT CONTRIBUTIONS

The major contributions of this project include:

- Development of a lightweight educational RTOS.
- Implementation of multitasking without using existing RTOS libraries.
- Design of a modular scheduler.
- Development of context switching mechanisms.

- Implementation of inter-task communication.
- Design of synchronization using semaphores.
- Development of reusable device drivers.
- Creation of a structured software architecture.
- Development of multiple embedded applications.
- Preparation of complete technical documentation.

11.5 ADVANTAGES OF THE DEVELOPED SYSTEM

The developed MiniRTOS provides several advantages compared to traditional bare-metal programming.

- Lightweight architecture
- Efficient multitasking
- Modular software design
- Portable implementation
- Reusable source code
- Low memory footprint
- Improved maintainability
- Better debugging capability
- Easy scalability
- Educational implementation of RTOS concepts

11.6 INDUSTRIAL APPLICATIONS

The developed MiniRTOS can be used in various embedded applications such as:

- Industrial Automation
- Smart Home Systems
- Automotive Electronics
- Medical Equipment
- Consumer Electronics
- Robotics
- Internet of Things (IoT)
- Wireless Sensor Networks
- Security Systems
- Environmental Monitoring
- Smart Agriculture
- Drone Controllers
- Smart Energy Systems
- Embedded Research Platforms

11.7 FUTURE SCOPE

Although the MiniRTOS provides all the essential kernel services required for multitasking, several advanced features can be incorporated in future versions.

Possible enhancements include:

ADVANCED SCHEDULING

- Round Robin Scheduling
- Earliest Deadline First (EDF)
- Rate Monotonic Scheduling (RMS)
- Multi-Level Queue Scheduling

ADVANCED SYNCHRONIZATION

- Mutexes
- Recursive Mutexes
- Event Flags
- Condition Variables

MEMORY MANAGEMENT

- Advanced Heap Manager
- Memory Pools
- Memory Protection Unit (MPU) Support
- Memory Fragmentation Reduction

COMMUNICATION

- Message Queues
- Mailboxes
- Event Notifications
- Shared Memory

NETWORKING

- TCP/IP Stack
- Ethernet Driver
- Wi-Fi Driver
- CAN Bus Support
- USB Device Support

FILE SYSTEM

- FAT File System
- SD Card Support
- Flash File System
- Data Logging

SECURITY

- Secure Boot
- Encryption
- Authentication
- Secure Firmware Update

POWER MANAGEMENT

- Tickless Scheduling
- Sleep Modes
- Dynamic Clock Scaling
- Low Power Operation

DEBUGGING

- Trace Viewer
- Kernel Debugger
- Runtime Statistics
- Performance Monitor

MULTI-CORE SUPPORT

- Symmetric Multi-Processing (SMP)
- Multi-Core Scheduling
- Processor Affinity
- Load Balancing

11.8 LEARNING OUTCOMES

This project helped in gaining practical knowledge in the following areas:

- ARM Cortex-M Architecture

- Embedded C Programming
- Real-Time Operating Systems
- Interrupt Handling
- Context Switching
- Device Driver Development
- Software Architecture
- Embedded Debugging
- STM32CubeIDE
- Version Control using Git
- GitHub Project Management
- Technical Documentation

11.9 CONCLUSION

The **ARM Cortex-M MiniRTOS and Device Driver Framework** project successfully demonstrates the implementation of a lightweight Real-Time Operating System capable of supporting multitasking on ARM Cortex-M microcontrollers. The project integrates a priority-based scheduler, task management, context switching, synchronization mechanisms, memory management, and modular device drivers into a cohesive software framework.

The successful execution of demonstration applications such as LED blinking, UART communication, producer-consumer synchronization, button interrupt handling, and multiple concurrent tasks validates the correctness and reliability of the implemented kernel and driver framework.

The modular architecture adopted throughout the project promotes maintainability, portability, and scalability, allowing future enhancements without significant redesign. By implementing the RTOS kernel from scratch, the project provides valuable insights into the internal working principles of operating systems, making it an excellent educational platform for students and a strong foundation for advanced embedded system development.

Overall, the developed MiniRTOS achieves its intended objectives and serves as a practical example of modern embedded software engineering practices. It demonstrates proficiency in embedded programming, operating system concepts, device driver development, and software architecture, making it suitable for academic projects, technical portfolios, GitHub repositories, and placement interviews.

REFERENCES

1. ARM Ltd., *ARM Cortex-M4 Processor Technical Reference Manual*, ARM Documentation.
2. Joseph Yiu, *The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors*, Newnes, 3rd Edition.
3. Richard Barry, *Using the FreeRTOS Real-Time Kernel*, Real Time Engineers Ltd.
4. Jonathan Valvano, *Embedded Systems: Real-Time Interfacing to ARM Cortex-M Microcontrollers*, Cengage Learning.
5. David Simon, *An Embedded Software Primer*, Addison-Wesley.
6. Steve Furber, *ARM System-on-Chip Architecture*, Pearson Education.
7. STMicroelectronics, *STM32 Reference Manual (RM0090/RM0368)*.
8. STMicroelectronics, *STM32 Datasheet*.
9. ARM CMSIS Documentation.
10. IEEE Xplore Digital Library – Research papers on RTOS and Embedded Systems.

APPENDICES

APPENDIX A

- Source Code Listings

APPENDIX B

- Scheduler Flowchart

APPENDIX C

- UML Diagrams

APPENDIX D

- Memory Map

APPENDIX E

- API Documentation

APPENDIX F

- Test Cases

APPENDIX G

- Output Screenshots

APPENDIX H

- GitHub Repository Structure

APPENDIX I

- Demo Video QR Code / Link

APPENDIX J

- User Manual

